

RESUMO DO MÓDULO

Mesma Rota com Métodos Diferentes

Como mencionamos anteriormente, uma mesma rota pode responder a métodos HTTP diferentes. Isso significa que podemos usar a mesma URL para executar diferentes operações, dependendo do método usado. Vamos explorar isso utilizando como exemplo a rota de usuário `/usuarios/:nome`.

Definindo a Rota para GET

Se criarmos uma rota GET como a seguir, conseguimos pegar o nome do usuário através do parâmetro e, por exemplo, buscar a informação dele em um banco de dados:

javascript

```
app.get('/usuarios/:nome', (req, res) => {  
  const nome = req.params.nome;  
  res.send(`Este é o perfil do usuário ${nome}`);  
});
```

Nesse caso, o método GET é usado para buscar informações do usuário na URL fornecida. Se você acessar `http://localhost:3000/usuarios/joao`, a resposta será:

arduino

Este é o perfil do usuário joao.

Definindo a Mesma Rota para PUT

Se quisermos atualizar os dados desse usuário, podemos usar a mesma rota, apenas trocando o método para PUT. Isso nos permite atualizar o perfil do usuário com a mesma URL:

javascript

```
app.put('/usuarios/:nome', (req, res) => {  
  const nome = req.params.nome;  
  res.send(`O perfil do usuário ${nome} foi atualizado com sucesso!`);  
});
```

Agora, ao usar a mesma URL (/usuarios/:nome) com o método PUT, a resposta será diferente, indicando que a atualização foi realizada com sucesso.

Testando no Thunder Client

Podemos testar essas configurações usando o Thunder Client (ou outras ferramentas como o Postman):

1. Requisição GET: Defina uma requisição do tipo GET para `http://localhost:3000/usuarios/joao` e veja a resposta retornando o perfil do usuário.
2. Requisição PUT: Defina uma requisição do tipo PUT para a mesma URL e observe a resposta indicando que o perfil foi atualizado.

Isso mostra como a mesma rota pode responder de maneiras diferentes, dependendo do método HTTP usado.

Usando Outros Métodos

Essa abordagem não se limita apenas a GET e PUT. Você pode definir a mesma rota para responder a POST e DELETE também. Por exemplo:

javascript

```
app.post('/usuarios/:nome', (req, res) => {  
  const nome = req.params.nome;  
  res.send(`O usuário ${nome} foi criado com sucesso!`);  
});  
  
app.delete('/usuarios/:nome', (req, res) => {  
  const nome = req.params.nome;  
  res.send(`O usuário ${nome} foi deletado com sucesso!`);  
});
```

Aqui, a rota `/usuarios/:nome` é usada para:

- POST: Criar um novo usuário.
- GET: Buscar as informações do usuário.
- PUT: Atualizar o perfil do usuário.
- DELETE: Excluir o usuário.

Conclusão

Usar a mesma rota com métodos diferentes é uma prática comum no desenvolvimento de APIs RESTful. Isso ajuda a manter o código mais organizado e a lógica centralizada, sem a necessidade de criar várias rotas diferentes para cada operação. Assim, você consegue gerenciar facilmente

operações CRUD (Criar, Ler, Atualizar, Deletar) em um único recurso, utilizando apenas métodos HTTP diferentes.
